University Timetable Scheduling System Using Particle Swarm Optimization (PSO)

Course: Evolutionary Computing / Artificial Intelligence
Submitted By: [Your Name]
Registration No: [Your Reg No]
Submitted To: [Teacher's Name]
Date: June 2026

Table of Contents

---

1. Introduction

University timetable scheduling is a well-known NP-hard combinatorial optimization problem. It involves assigning courses, teachers, rooms, and time slots in such a way that no conflicts arise while satisfying multiple hard and soft constraints. Manual scheduling is tedious, error-prone, and becomes infeasible as the number of resources grows.

This project presents an automated timetable scheduling system that employs **Particle Swarm Optimization (PSO)** — a population-based metaheuristic algorithm inspired by the social behavior of bird flocking and fish schooling. The system provides a full-stack web application where users can manage academic resources (courses, teachers, rooms, timeslots) and generate optimized, clash-free timetables with a single click.

---

2. Problem Statement

Given a set of:

- **Courses** (e.g., CS101, CS102, ...)
- **Teachers** (e.g., Dr. Alan Turing, Dr. Sara Ali, ...)
- **Rooms** (e.g., Room A-101, Lab B-201, ...)
- **Timeslots** (e.g., Monday 09:00–10:00, Tuesday 14:00–15:00, ...)

The goal is to assign each course to exactly one teacher, one room, and one timeslot such that:

Constraint Type	Description	Penalty
Hard	No two courses share the same teacher at the same time	+10
Hard	No two courses share the same room at the same time	+10
Soft	Avoid back-to-back classes for the same teacher	+2
Soft	Distribute classes evenly across days of the week	+1

---

3. Objectives

1. Implement a **Discrete Particle Swarm Optimization** algorithm to solve the timetable scheduling problem.
2. Develop a **full-stack web application** with CRUD operations for managing courses, teachers, rooms, and timeslots.
3. Provide a **before-vs-after comparison** showing a random (unoptimized) timetable alongside the PSO-optimized result.
4. Visualize the **convergence behavior** of the algorithm through a real-time fitness chart.
5. Highlight scheduling **clashes** with clear visual indicators (color coding).

---

4. Literature Review

4.1 What is PSO?

Particle Swarm Optimization was introduced by **Kennedy and Eberhart in 1995**. It is inspired by the collective intelligence observed in bird flocks and fish schools. A "swarm" of candidate solutions (called particles) moves through the search space, influenced by:

- **Personal experience** (pBest) — the best solution each particle has found so far.
- **Social influence** (gBest) — the best solution found by any particle in the swarm.

4.2 PSO vs. Other Metaheuristics

Algorithm	Inspiration	Operators	Suitability for Scheduling
Genetic Algorithm (GA)	Natural selection	Crossover, Mutation, Selection	High
Particle Swarm Optimization (PSO)	**Bird flocking**	**Velocity update, Position update**	**High**
Simulated Annealing (SA)	Metal cooling	Temperature-based acceptance	Moderate
Ant Colony Optimization (ACO)	Ant foraging	Pheromone trails	High

4.3 Why PSO for This Project?

- **Fewer parameters** to tune compared to GA (no crossover/mutation rates — just w, c1, c2).
- **Faster convergence** due to information sharing through gBest.

- **Simpler implementation** — no complex selection or crossover operators.
- Well-suited for **combinatorial optimization** when adapted to discrete spaces.

---

5. Methodology — Particle Swarm Optimization

5.1 Standard PSO Equations

The standard PSO velocity and position update equations are:

```
v(t+1) = w * v(t) + c1 * r1 * (pBest - x(t)) + c2 * r2 * (gBest - x(t))
x(t+1) = x(t) + v(t+1)
```

Where:

- `v(t)` = velocity of the particle at iteration t
- `x(t)` = position (solution) of the particle at iteration t
- `w` = inertia weight (controls exploration vs exploitation)
- `c1` = cognitive coefficient (attraction toward personal best)
- `c2` = social coefficient (attraction toward global best)
- `r1, r2` = random numbers in [0, 1]
- `pBest` = personal best position of the particle
- `gBest` = global best position found by any particle

5.2 Discrete PSO Adaptation

Since timetable scheduling is a **discrete combinatorial** problem (we can't have fractional room or teacher assignments), we adapt PSO using a **sigmoid-based probability approach**:

1. **Velocity** represents the **probability of changing** a gene dimension (teacher, room, or timeslot).
2. The velocity is passed through a **sigmoid function**: `σ(v) = 1 / (1 + e^(-v))`
3. If `random() < σ(v)`, the particle updates that dimension by moving toward either pBest or gBest.
4. The decision to follow pBest vs gBest is weighted by the ratio `c2 / (c1 + c2)`.

5.3 Hyperparameters Used

Parameter	Symbol	Value	Purpose
Inertia Weight	w	0.7	Controls momentum; prevents sudden changes
Cognitive Coefficient	c1	1.5	Strength of personal best attraction
Social Coefficient	c2	1.5	Strength of global best attraction
Swarm Size	—	30	Number of particles in the swarm
Max Iterations	—	100	Maximum optimization cycles
Velocity Clamp	—	[-4, 4]	Prevents velocity explosion
Random Exploration Rate	—	5%	Prevents premature convergence

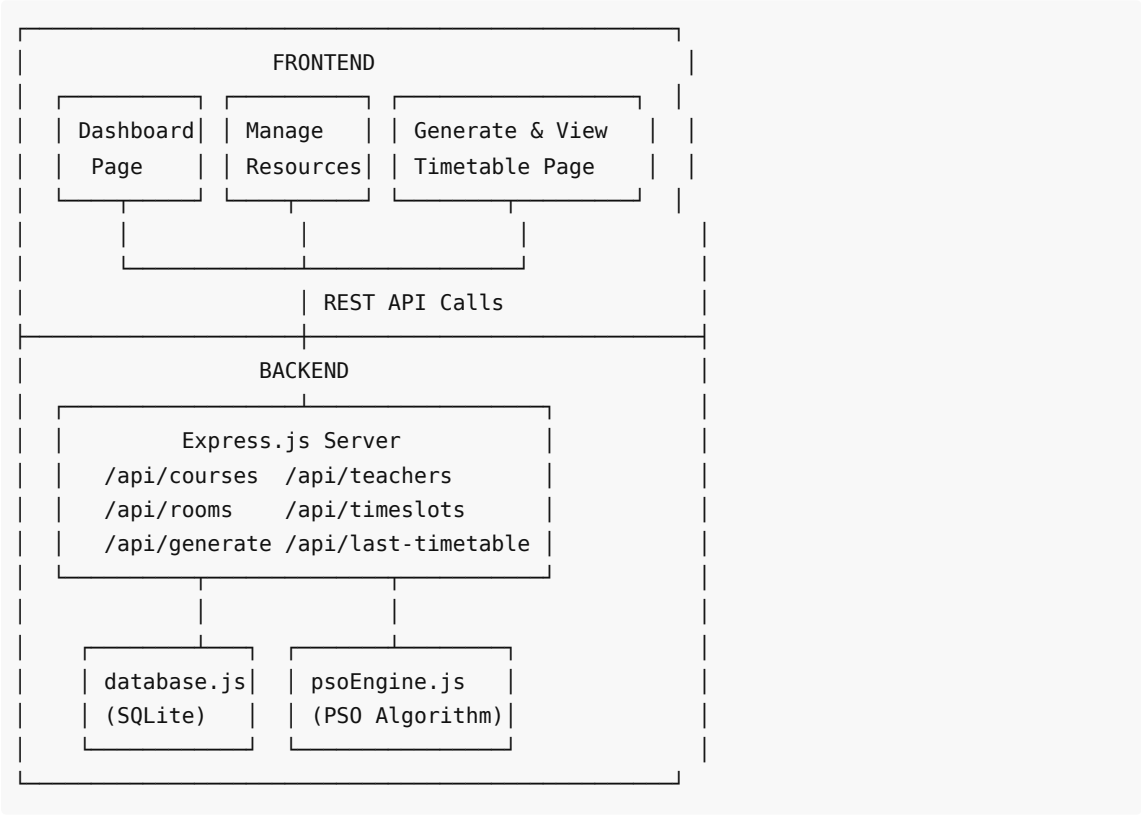
---

6. System Architecture

6.1 Technology Stack

Layer	Technology	Purpose
Backend	Node.js + Express.js	REST API server
Database	SQLite (via sql.js)	Persistent data storage
Frontend	HTML + CSS + Vanilla JavaScript	User interface
Charts	Chart.js	Fitness convergence visualization
Deployment	Vercel-compatible	Serverless deployment support

6.2 Architecture Diagram



6.3 Project File Structure

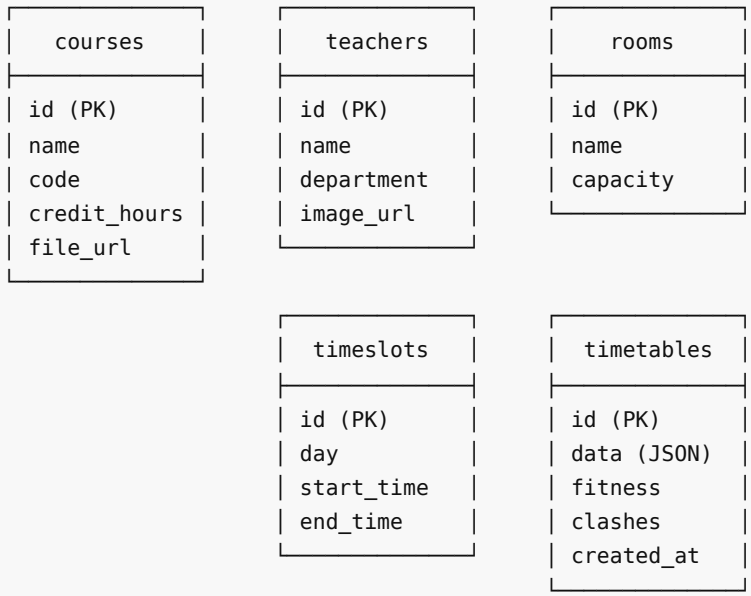
```
timetable-ga/
├── server.js          → Express server & API routes
├── psoEngine.js       → PSO algorithm implementation (core)
├── database.js        → SQLite database layer & CRUD
├── package.json       → Dependencies & scripts
├── timetable.db       → SQLite database file
├── data.csv           → Sample data for CSV import
├── public/
```

```
│   ├── index.html     → Landing page
│   ├── dashboard.html → Resource management (CRUD)
│   ├── generate.html  → Timetable generation & results
│   ├── about.html     → About page
│   ├── blog.html      → Blog/documentation page
│   ├── css/style.css  → All styles (dark theme, glassmorphism)
│   └── js/app.js      → Frontend logic & API calls
```

---

7. Database Design

7.1 Entity-Relationship Diagram



7.2 Sample Data

The system seeds **6 courses**, **4 teachers**, **4 rooms**, and **20 timeslots** (5 days × 4 time periods) automatically on first run.

---

8. PSO Engine — Detailed Implementation

8.1 Particle Representation

Each **particle** represents one complete timetable. A particle's **position** is an array of gene objects:

```
Particle Position = [Gene₁, Gene₂, ..., Geneₙ]

Where each Gene = {
    course:   Course object (e.g., CS101)
    teacher:  Teacher object (e.g., Dr. Alan Turing)
    room:     Room object (e.g., Room A-101)
    timeslot: Timeslot object (e.g., Monday 09:00-10:00)
}
```

Each gene has **3 mutable dimensions**: teacher, room, and timeslot. The course is fixed per gene.

8.2 Velocity Representation

Each gene has a velocity vector with three probability values:

```
Velocity[g] = {
    teacher:  float in [-4, 4]
    room:     float in [-4, 4]
    timeslot: float in [-4, 4]
}
```

Initial velocities are randomized in [0, 0.5] to start with moderate exploration.

8.3 Algorithm Flowchart

```
START
  │
  ▼
[1] Initialize swarm of 30 particles with random positions
  │
  ▼
[2] Evaluate fitness of each particle
  │
  ▼
[3] Set each particle's pBest = current position
    Set gBest = best particle in swarm
  │
  ▼
[4] ──────── ITERATION LOOP (max 100) ────────
  │                                           │
  ▼                                           │
[5] For each particle, for each gene:         │
    • Update velocity using PSO equation      │
    • Clamp velocity to [-4, 4]               │
    • Apply sigmoid → get probability         │
    • If random() < probability:              │
      - Move toward pBest or gBest            │
    • 5% chance: random exploration           │
  │                                           │
  ▼                                           │
[6] Evaluate fitness (update pBest if better) │
  │                                           │
  ▼                                           │
[7] Update gBest if any particle improved     │
  │                                           │
  ▼                                           │
[8] Record fitness history for chart          │
  │                                           │
  ▼                                           │
[9] If fitness == 1.0 AND iter > 20 → STOP    │
    Else → go to [4]                          │
```

```
  │                                              │
  ◄──────────────────────────────────────────────┘
  │
  ▼
[10] Return best timetable, fitness, clashes, history
  │
  ▼
END
```

8.4 Key Code Walkthrough

8.4.1 Velocity Update (Lines 198–212 of psoEngine.js)

```
// For each dimension (teacher, room, timeslot):
const cognitivePull = (pBestId !== currentId) ? 1 : 0;
const socialPull    = (gBestId !== currentId) ? 1 : 0;

// PSO velocity equation
velocity = w * velocity + c1 * r1 * cognitivePull + c2 * r2 * socialPull;

// Clamp to prevent explosion
velocity = Math.max(-4, Math.min(4, velocity));
```

Explanation: If the particle's current value differs from its personal best or the global best, a "pull" of magnitude 1 is generated. This pull, weighted by `c1` / `c2` and random factors `r1` / `r2`, updates the velocity. The inertia weight `w` preserves momentum from the previous iteration.

8.4.2 Position Update via Sigmoid (Lines 218–237)

```
const prob = sigmoid(velocity);  // Convert velocity → probability

if (Math.random() < prob) {
    const moveToGlobal = Math.random() < (c2 / (c1 + c2));

    if (moveToGlobal) {
        // Copy value from global best
        position[g][dim] = gBest[g][dim];
    } else {
        // Copy value from personal best
        position[g][dim] = pBest[g][dim];
    }
}
```

Explanation: The sigmoid converts velocity (which can be any real number) into a probability between 0 and 1. Higher velocity = higher probability of changing. When a change occurs, the particle moves toward either gBest (with probability `c2/(c1+c2) = 0.5`) or pBest.

8.4.3 Random Exploration (Lines 241–249)

```
if (Math.random() < 0.05) {
    const dim = ['teacher', 'room', 'timeslot'][Math.floor(Math.random() * 3)];
    const pool = dim === 'teacher' ? teachers :
                 dim === 'room' ? rooms : timeslots;
    position[g][dim] = pool[Math.floor(Math.random() * pool.length)];
}
```

Explanation: A 5% random mutation prevents all particles from converging to the same solution too early (premature convergence). This maintains diversity in the swarm.

---

9. Fitness Function Design

9.1 Formula

```
Fitness = 1 / (1 + Total Penalty)
```

- **Perfect timetable** (0 penalties) → Fitness = **1.0**
- **Bad timetable** (many penalties) → Fitness → **0.0**

9.2 Penalty Breakdown

#	Constraint	Type	Penalty	Detection Logic
1	Teacher clash (same teacher, same timeslot)	Hard	+10	Compare all gene pairs
2	Room clash (same room, same timeslot)	Hard	+10	Compare all gene pairs
3	Back-to-back classes for same teacher	Soft	+2	Check consecutive time hours on same day
4	Uneven day distribution	Soft	+1	Standard deviation from average classes/day

9.3 Personal Best Update

After each fitness evaluation, if the new fitness exceeds the particle's historical best (`pBestFitness`), the current position is saved as the new `pBest` . This ensures particles always remember their best-ever solution.

---

10. User Interface

10.1 Pages

Page	URL	Description
Landing Page	`/`	Hero section, feature highlights, call-to-action
Dashboard	`/dashboard.html`	CRUD management for courses, teachers, rooms, timeslots

Generate	`/generate.html`	Run PSO, view fitness chart, before vs after comparison
About	`/about.html`	Project and algorithm information
Blog	`/blog.html`	Documentation and PSO explanation

10.2 Design Features

- **Dark theme** with glassmorphism effects (frosted glass cards)
- **Gradient accents** using cyan/purple color palette
- **Responsive layout** for desktop and mobile
- **Real-time fitness chart** (Chart.js) showing convergence over iterations
- **Color-coded clash indicators**: Red for clashes, green for clash-free
- **Before vs After view**: Side-by-side comparison of random vs optimized timetable

---

11. Results & Analysis

11.1 Test Configuration

Parameter	Value
Courses	6
Teachers	4
Rooms	4
Timeslots	20 (5 days × 4 periods)
Swarm Size	30
Max Iterations	100

11.2 Typical Performance

Metric	Random Timetable	PSO-Optimized
Fitness Score	0.03 – 0.10	0.90 – 1.00
Teacher Clashes	5 – 15	0 – 1
Room Clashes	3 – 10	0
Convergence	N/A	15 – 40 iterations

11.3 Convergence Behavior

The fitness chart typically shows:

1. **Rapid initial improvement** (iterations 1–10): Particles quickly find better regions.
2. **Gradual refinement** (iterations 10–30): Fine-tuning of assignments.
3. **Plateau at optimal** (iterations 30+): gBest stabilizes near or at 1.0.

11.4 Why PSO Works Well Here

- The **social component (gBest)** allows good partial solutions to spread quickly through the swarm.
- The **cognitive component (pBest)** prevents particles from losing good configurations they've found.
- The **5% random exploration** maintains diversity and helps escape local optima.
- The **sigmoid probability mechanism** provides a natural way to handle discrete variables.

---

12. Conclusion & Future Work

12.1 Conclusion

This project successfully demonstrates the application of Particle Swarm Optimization to the university timetable scheduling problem. The PSO algorithm consistently produces clash-free or near-optimal timetables within 100 iterations, significantly outperforming random assignment. The web-based interface provides an intuitive way to manage resources and visualize optimization results.

12.2 Future Work

- **Multi-objective optimization**: Consider student preferences and room proximity.
- **Hybrid PSO-GA**: Combine PSO with genetic operators for improved exploration.
- **Constraint customization**: Allow users to define custom hard/soft constraints.
- **Export functionality**: Generate PDF/Excel timetable outputs.
- **Real-time collaboration**: Multi-user support for department-level scheduling.

---

13. References

1. Kennedy, J. & Eberhart, R. (1995). "Particle Swarm Optimization." *Proceedings of IEEE International Conference on Neural Networks*, Vol. 4, pp. 1942–1948.
2. Shi, Y. & Eberhart, R. (1998). "A Modified Particle Swarm Optimizer." *IEEE International Conference on Evolutionary Computation*, pp. 69–73.
3. Clerc, M. & Kennedy, J. (2002). "The Particle Swarm – Explosion, Stability, and Convergence." *IEEE Transactions on Evolutionary Computation*, Vol. 6, No. 1, pp. 58–73.
4. Al-Betar, M.A. (2021). "University Course Timetabling Using a Hybrid Harmony Search Metaheuristic Algorithm." *IEEE Transactions on Systems, Man, and Cybernetics*.
5. Colorni, A., Dorigo, M., & Maniezzo, V. (1992). "A Genetic Algorithm to Solve the Timetable Problem." *Computational Optimization and Applications*.

> ***Note:*** *This report was prepared for academic purposes. The complete source code is available in the project repository.*